Here I wrote the prompt, which is essentially telling the model to write a question in the form that I want, which is to have incorrect answers that are very realistic and appear to be correct.

I also tell it to explain each answer choice upon the user answering.

ROLE: You are creating diagnostic learning questions for CS182 (Deep Learning).

Write exactly 1 multiple-choice question about the concept: Optimizers
Include:
- Exactly 1 correct answer
- Exactly 3 distractors that each contain a subtle, realistic misconception
- Avoid obviously incorrect or irrelevant distractors
- Do NOT give away the correct answer

Label answers A–D clearly.

Once the user responds with an answer, inform if it was correct or not, and then explain the reasonings behind each answer. For the correct one explain why it is correct, and for the incorrect ones explain the subtle error or misconception that leads to it.

Here's your diagnostic question:

Question: Optimizers

This was a good first question, as on first glance, these all seem like plausible things to happen.

Which of the following best describes why Adam often converges faster than vanilla SGD in training deep neural networks?

A. Adam uses full second-order curvature information to compute exact Newton steps.
B. Adam adapts individual learning rates for each parameter using estimates of first and second moments of gradients.
C. Adam eliminates the need for learning rate tuning because it automatically finds the globally optimal step size.
D. Adam guarantees convergence to the global minimum by reducing gradient variance to zero.

Please answer with A, B, C, or D.

C

Thanks for your answer!

Your answer: C — ❌ *Not correct.*

Correct Answer: B

Explanation of All Options

A. Adam uses full second-order curvature information to compute exact Newton steps.
❌ *Subtle misconception:*
Adam does **not** compute curvature like Newton's method. It only uses gradient history (first and second **moments**, not derivatives).

The model does what is expected, by explaining all the choices, and saying what misconceptions arise from each wrong answer.

B. Adam adapts individual learning rates for each parameter using estimates of first and second moments of gradients.

✅ *Correct:*

Adam keeps per-parameter estimates of the **mean** and **uncentered variance** of gradients, leading to adaptive effective learning rates that improve convergence speed.

C. Adam eliminates the need for learning rate tuning because it automatically finds the globally optimal step size.

❌ *Subtle misconception:*

Adam still needs learning rate selection (α matters!), and it does **not** guarantee an optimal global step size.

D. Adam guarantees convergence to the global minimum by reducing gradient variance to zero.

❌ *Subtle misconception:*

There is **no global convergence guarantee** in general non-convex settings like deep networks.

If you'd like, I can give you another question targeting a different optimizer concept—or progressively increase difficulty!

Do another optimizer question

Here's your next diagnostic question:

Question: Optimizers

Why is momentum often helpful when training deep neural networks with SGD?

A. It increases the learning rate automatically whenever gradients become small, guaranteeing faster convergence.
B. It helps smooth out noisy gradient directions by accumulating a velocity term, reducing oscillations in sharp curvature regions.
C. It replaces the need for gradient descent entirely by directly estimating the optimal weights from previous updates.
D. It prevents overfitting by penalizing large weight updates through L2 regularization built into its update rule.

Please answer with A, B, C, or D.

This question was a little easier in my opinion, but again all the answers seem plausible, and during a stressed out exam state of mind, I could imagine thinking of all of these as possibly correct.

B

Your answer: B — ✅ Correct!

Why B is correct:

B. It helps smooth out noisy gradient directions by accumulating a velocity term, reducing oscillations in sharp curvature regions.

✔ Momentum builds on past gradients (a *velocity vector*) so updates are more stable and directional. This helps accelerate progress along consistent directions and damp oscillations in steep or noisy areas.

Why the others are not correct:

A. It increases the learning rate automatically whenever gradients become small…
❌ Momentum does not change the base learning rate — it only smooths updates through accumulated gradients.

C. It replaces the need for gradient descent entirely…
❌ Momentum is **part of** gradient descent, not a replacement.

D. It prevents overfitting via built-in L2 regularization…
❌ That describes **weight decay**, not momentum.

Would you like another optimizer question — perhaps a bit more challenging?

more challenging

Alright — stepping it up!

Challenging Question: Optimizers

Which scenario most clearly highlights a disadvantage of using Adam compared to SGD with momentum?

A. Training a very deep network with sparse gradients, where per-parameter adaptivity is needed for progress.
B. Optimizing a non-convex loss where Adam may converge to sharp minima that generalize worse than the flatter minima SGD can reach.
C. Training on a dataset with significant label noise, where Adam's bias correction leads to unstable updates.
D. Using a very small batch size, where Adam cannot reduce gradient variance and diverges quickly.

Please answer with A, B, C, or D.

This again is a hard question, with the answer choices being hard to discern the correctness of. In fact, I did get this question wrong, which was quite insightful.

The model again did explain all the answer choices as required.

A

Your answer: A — ❌ Not correct.

Correct Answer: B

Here's the reasoning:

B. Adam may converge to sharp minima that generalize worse than the flatter minima SGD can reach.

✔ This is a well-known trade-off:

Adam often **converges faster**, but its adaptive updates may lead to **sharp** solutions with poorer generalization — especially compared to SGD with momentum, which tends to drift toward **flatter minima**.

Why the others are incorrect (subtle issues):

A. Training a very deep network with sparse gradients…

✗ This is actually where Adam *shines* — adaptivity helps parameters with rare gradients (like embeddings).

C. Label noise causes Adam's bias correction to be unstable.

✗ Adam handles noisy gradients better than plain SGD in many cases; bias correction isn't uniquely sensitive to label noise.

D. Small batch size causes Adam to diverge.

✗ Adam is relatively robust with small batches — variance resilience is one of its strengths.

Would you like an **even harder** question — maybe one involving learning rate schedules or convergence theory?

Can you do a more math question (not like math calculation but using possible lienar algebra or calculus/ derivations)

I asked it to switch modalities to a more derivation question, and it gave a reasonable seeming question that wouldn't be out of place in a discussion or homework in my opinion.

Thought for 7s

Here's a more mathy optimizer question for you:

Question: Optimizers & Quadratic Objectives

Consider gradient descent on a quadratic loss

$$f(w) = \frac{1}{2} w^\top A w$$

where $A \in \mathbb{R}^{d \times d}$ is symmetric positive definite with eigenvalues

$$0 < \lambda_{\min} \le \lambda_2 \le \cdots \le \lambda_{\max}.$$

We use the update

$$w_{t+1} = w_t - \eta \nabla f(w_t).$$

Which condition on the constant step size η guarantees that gradient descent **converges to the minimum for any initialization** w_0?

A. $0 < \eta < \frac{1}{\lambda_{\min}}$

B. $0 < \eta < \frac{1}{\lambda_{\max}}$

C. $0 < \eta < \frac{2}{\lambda_{\max}}$

D. $0 < \eta < \frac{2}{\lambda_{\min}}$

Answer with **A, B, C, or D**. Then I'll tell you if you're correct and walk through the reasoning behind each option.

C

Nice, going for the mathy one 😄

Your answer: C — ✅ Correct!

Big picture idea

It gives a good mathematical explanation for the correct answer, as well as quick reasonings as to why someone may have chosen the other answers and why those were wrong.

For gradient descent on

$$f(w) = \tfrac{1}{2} w^\top A w, \quad A > 0,$$

the update is

$$w_{t+1} = w_t - \eta A w_t = (I - \eta A) w_t.$$

Convergence for **any** w_0 happens iff the spectral radius of $I - \eta A$ is < 1, i.e.

$$|1 - \eta \lambda_i| < 1 \quad \text{for all eigenvalues } \lambda_i.$$

The worst case is $\lambda_{\max}$, which gives the constraint

$$-1 < 1 - \eta \lambda_{\max} < 1 \Rightarrow 0 < \eta < \frac{2}{\lambda_{\max}}.$$

So **C** is the right condition.

Option-by-option breakdown

A. $0 < \eta < \dfrac{1}{\lambda_{\min}}$

❌ Uses the *smallest* eigenvalue.

- This is **too loose in the wrong direction**: the largest eigenvalue controls stability.
- You could have $\eta < 1/\lambda_{\min}$ but still $\eta > 2/\lambda_{\max}$, causing divergence.

B. $0 < \eta < \dfrac{1}{\lambda_{\max}}$

❌ Too conservative.

- This is actually a *sufficient* condition for convergence (since $1/\lambda_{\max} < 2/\lambda_{\max}$),
- but it's **not the tight condition** the question is asking for; it's stricter than needed.

C. $0 < \eta < \dfrac{2}{\lambda_{\max}}$

✅ **Correct.**

- Directly comes from requiring $|1 - \eta \lambda_i| < 1$ for all eigenvalues.
- This is the **necessary and sufficient** range for convergence for all initializations.

D. $0 < \eta < \dfrac{2}{\lambda_{\min}}$

❌ Completely wrong scale.

- Because $\lambda_{\min} \le \lambda_{\max}$, we have $2/\lambda_{\min} \ge 2/\lambda_{\max}$.
- This allows step sizes that are **too large** for the largest eigenvalue direction → divergence.

If you want, next we can do a derivation-style question about **momentum on a 1D quadratic** or about **Adam's effective step size** in one coordinate.

Switch to attentions and transformers

I had it switch topics, which it did very easily, and generated another mathematical type question, this time with regards to attention and transformers.

Great — switching domains!

Here's a more math-oriented Transformer/Attention question:

Question: Self-Attention Scaling

In scaled dot-product attention, the attention scores between query $q_i \in \mathbb{R}^d$ and key $k_j \in \mathbb{R}^d$ are computed as:

$$\text{score}(i, j) = \frac{q_i^\top k_j}{\sqrt{d}}$$

This is a sort of math question, but also conceptual, which was interesting as the model was blending types of questions together. This is good though, and I could see how someone could perhaps mix up answer choices B and D, as both ensure the score is not too big.

Why is the division by $\sqrt{d}$ necessary?

A. Without scaling, the dot products shrink toward zero as d grows, causing vanishing gradients in softmax.
B. Scaling keeps the softmax logits in a stable range because unscaled dot products grow in magnitude with d, preventing extremely peaked attention distributions.
C. Scaling normalizes the queries and keys to unit length, ensuring that attention weights sum to one.
D. Scaling makes attention invariant to linear transformations of the embedding dimension, ensuring rotational symmetry in feature space.

Please answer with **A, B, C, or D**.

B

Your answer: B — ✅ Correct!

Why B is correct

Dot products of two random d-dimensional vectors have variance proportional to d. As d increases, the logits into softmax become **larger in magnitude**, making softmax outputs **extremely sharp** — gradients nearly vanish because one token dominates.

Dividing by $\sqrt{d}$ keeps the logits roughly in a stable variance range → better training dynamics.

Why the others are subtly wrong

A. Dot products shrink toward zero as d grows

❌ Opposite: dot products typically **grow**, not shrink. So scaling is not to prevent vanishing logits.

C. Scaling normalizes queries/keys to unit length

❌ Normalization would require dividing by norms $\|q_i\|\|k_j\|$, not $\sqrt{d}$. This doesn't ensure unit vectors.

D. Scaling makes attention invariant to linear transforms

❌ Linear invariance is not achieved by scalar division. The motivation is statistical scaling, not geometric symmetry.

It again explains all the choices, and interesetingly, without prompting, gives me more choices of what "types" of questions I would like to have.

If you'd like, the next one can involve **multi-head attention**, **residual connections**, or **transformer expressivity** — pick your challenge:

A) Derivation / math-based

B) Architectural reasoning

C) Optimization & training stability

D) Theory (expressive power, universality)

Which direction should we go?

do a more conceptual question

Got it — here's a higher-level conceptual reasoning question about attention mechanisms:

I again switched the modality of the question back to more conceptual, and it gave a good question with answers that could seem correct. In particular, I did think for a second that A could be correct, but ultimately I did answer B.

Question: Multi-Head Attention

Why does multi-head attention improve model performance compared to using a single attention head with the same total dimensionality?

A. Multi-head attention reduces computation by applying attention in parallel, lowering FLOPs while keeping expressivity the same.
B. Multi-head attention allows the model to jointly attend to different types of relationships by projecting queries/keys/values into multiple learned subspaces.
C. Multi-head attention guarantees orthogonal attention patterns across heads, increasing the diversity of learned features.
D. Multi-head attention ensures that attention weights are always uniformly distributed, preventing over-fitting to any single token.

Answer with **A, B, C, or D**.

B

Your answer: B — ✅ Correct!

Why B is correct

Multi-head attention **splits** the embedding into multiple subspaces and learns separate projections for each head.

Each head can then specialize in different relationship types — positional cues, syntactic structure, long-range dependencies, etc.

This gives richer representational power than a single head with the same overall dimension.

Why the others are subtly wrong

A. Multi-head attention reduces computation

❌ It actually *increases* computation — multiple projections and attentions in parallel add cost.

C. Heads are guaranteed orthogonal

❌ There is no constraint enforcing orthogonality; heads can collapse and attend similarly (a known failure mode).

D. Ensures uniform attention weights

❌ Attention can still be highly peaked — nothing enforces uniformity.

If you want, I can now give you a **conceptual + math** hybrid question about:

1️⃣ Residual connections and gradient flow
2️⃣ Positional encodings and sinusoid properties
3️⃣ Transformer expressivity vs. RNNs
4️⃣ Causal vs. bidirectional attention and self-supervision

Which one sounds good?

For this last part, I tested one last modality, which was a code writing question, to see how the model would perform on this task.

now do a code filling question

Perfect — here's a **code-filling** question focused on attention mechanics. The goal is to test conceptual correctness, not just syntax.

Question: Fill in the Missing Code — Scaled Dot-Product Attention

The function below should compute scaled dot-product attention:

python Copy code

```python
def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q, K, V: shape (batch, heads, seq_len, d_k)

    # 1️⃣ Compute raw attention scores
    scores = ____________________________

    # Apply optional mask (assume mask shape is broadcastable)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # 2️⃣ Softmax over the key dimension
    attn = torch.softmax(_________________, dim=-1)

    # 3️⃣ Weighted sum of values
    output = ____________________________
```

It gave a very reasonable looking fill-in-the-blank coding question that could be right out of the homework or 61A type exams.

```
        return output, attn
```

Choose the correct set of replacements:

A.

python Copy code

```
torch.matmul(Q, K) / d_k
scores
torch.matmul(attn, V)
```

B.

python Copy code

```
torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
scores
torch.matmul(attn, V)
```

C.

python Copy code

```
torch.matmul(Q, K.transpose(-2, -1)) * math.sqrt(d_k)
attn
torch.matmul(V, attn)
```

D.

python Copy code

```
torch.matmul(K, Q.transpose(-2, -1)) / math.sqrt(d_k)
scores
torch.matmul(attn, V.transpose(-2, -1))
```

Respond with **A, B, C, or D** — then I'll break down why.

D

Your answer: D — ❌ Not correct.

Correct Answer: B

Here's why:

Check each step

1️⃣ Attention score computation

We need:

- dot product between **Q and K**
- transpose K to align dimensions: $Q @ K^\top$

- scale by $\sqrt{d_k}$

So:

```python
torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
```

This matches only **Option B**.

Once I answered wrong, the model explained each of the code snippets, and stating where the code went wrong. This is helpful as it localizes where the understanding went wrong.

2️⃣ Softmax

Softmax must operate over the **key** dimension (last axis in this shape):

```python
torch.softmax(scores, dim=-1)
```

Option B uses `scores` correctly.

3️⃣ Weighted sum of V

We combine attention weights (batch, heads, seq_len, seq_len) with values:

```python
torch.matmul(attn, V)
```

Correct in B.

Why the others are wrong

Option	Issue
A	Missing K transpose + wrong scaling
C	Multiplies instead of divides by √d_k, swaps order in matmul
D	Reverses Q and K, wrong matmul for output

Would you like another code question — maybe **multi-head attention assembly**, **causal masking**, or **residual + layer norm** ordering?